in order to call to `compare_exchange_strong()` on the `data` member of the node ❻, you call a new function `increase_external_count()` to increase the count ❺, and then afterward you call `free_external_counter()` on the old tail value ❼.

With the `push()` side dealt with, let's take a look at `pop()`. This is shown in the following listing and blends the reference-counting logic from the `pop()` implementation in listing 7.11 with the queue-pop logic from listing 7.13.

Listing 7.16. Popping a node from a lock-free queue with a reference-counted tail

```
template<typename T>
class lock_free_queue
{
private:
    struct node
    {
        void release_ref();
    };
public:
    std:
    {
                    T* const res=ptr->data.exchange(nullptr);
                    free_external_counter(old_head);          <-5
                    return std::unique_ptr<T>(res);
                }
                ptr->release_ref();     <-6
                                                              ); <-1
            }
        }
    };
        node* const ptr=old_head.ptr;
        if(ptr==tail.load().ptr)
        {

            ptr->release_ref();          <-3
            return std::unique_ptr<T>();
        }
        if(head.compare_exchange_strong(old_head,ptr->next))     <-4
        {
            T* const res=ptr->data.exchange(nullptr);
            free_external_counter(old_head);          <-5
            return std::unique_ptr<T>(res);
        }
        ptr->release_ref();     <-6
    }
};
```